XentiQ 1R Technical Documentation

System Overview

This repository contains the source code for the Arduino controller for the XentiQ 1R machine, which communicates with the PIC microcontroller via UART. The Arduino controller is responsible for the following:

- Front-end display and user interface (using the EVE display)
- Stepper motor control and limit switch monitoring
- Profile management
- PIC microcontroller communication
- PLC communication

System Architecture

The codebase is organized into the following modules:

Module	Description
`controllers/`	State machine controllers for different UI screens and workflows
`views/`	Display rendering components for UI elements
`hardware/`	Low-level hardware interfaces (motors, limit switches)
`serial/`	UART communication protocols for PIC interface
`logic/`	Business logic for profile handling and dispensing operations
`gpu/`	Graphics processing utilities for the EVE display

Configuration

System configuration is centralized in **`Config.h`**, which contains all hardware and operational parameters:

Key Configuration Parameters

- **Motor Settings**: Speed, acceleration, and steps-per-unit for X/Y/Z axes
- **Limit Switch Pins**: Hardware pin assignments for safety limits
- **Dispenser Timeouts**: ACK timeout, cycle timeout
- **Profile Defaults**: Origin position, pitch spacing, Z-dip depth, cycle count
- **Security**: Password settings for system access
- **Debug Flags**: Enable/disable logging, screen-only mode, memory monitoring

System States

This document provides visual references for the different states of the system. Most states represent a single `Controller`, which handle the system's logic and state. For simplicity, the "Settings" state diagram is separated from the main state diagram.

Main States

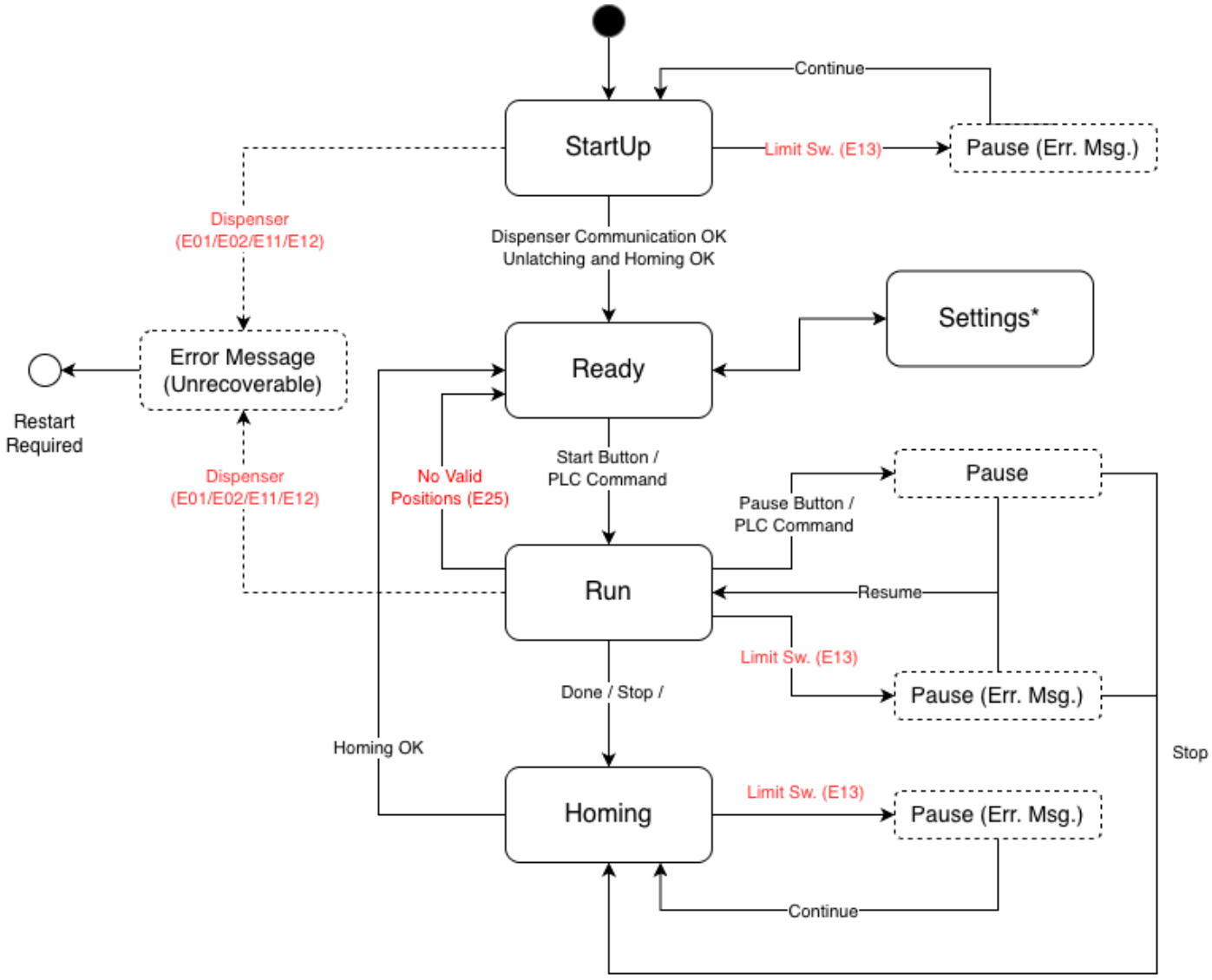


Figure 1: States Diagram

Settings States

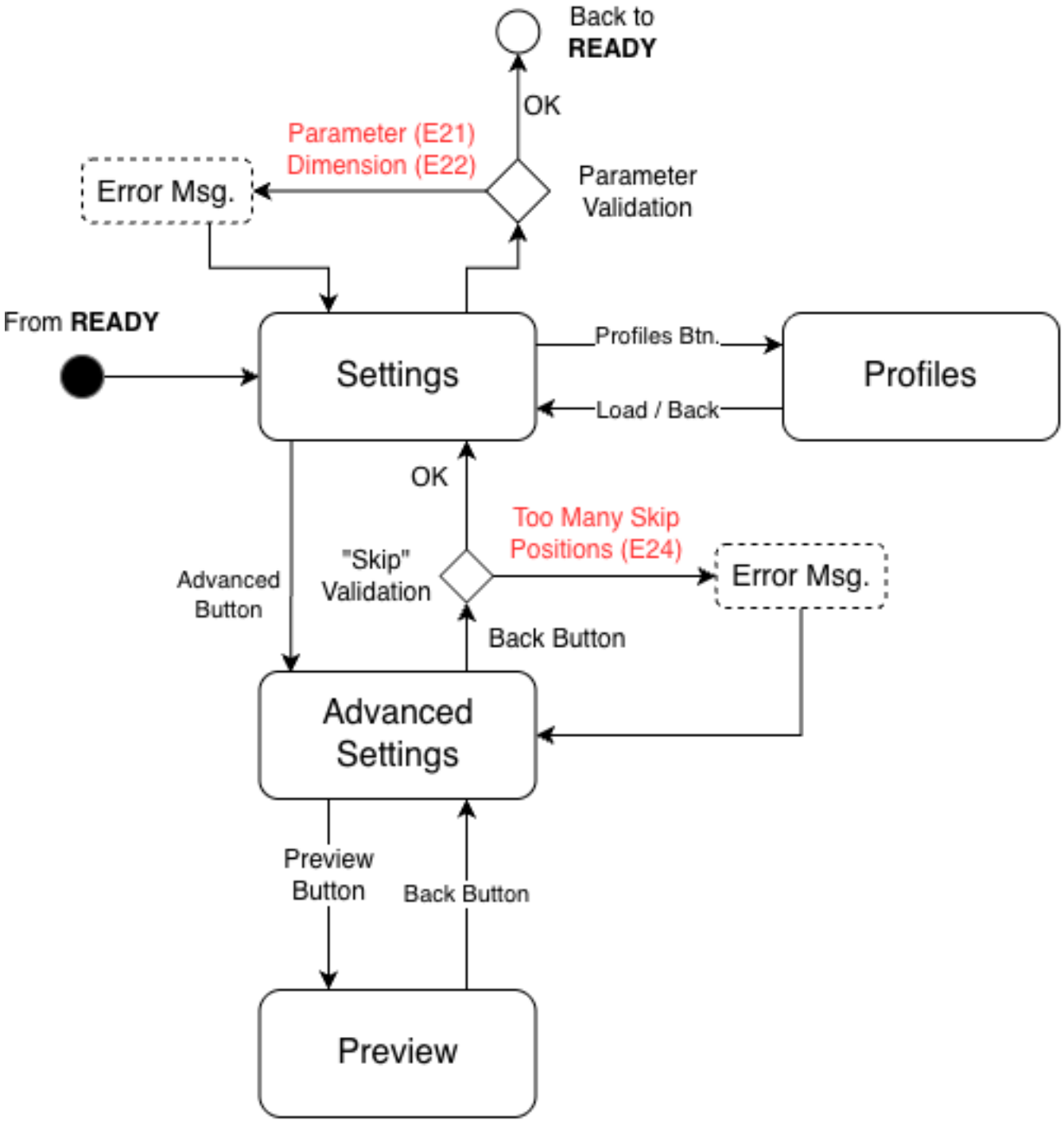


Figure 2: Settings States Diagram

Error Codes

This document lists all error codes and their meanings.

Error Reference Table

Error Code	Explanation
E01 - IR Sensor Error	The system failed to detect the dispenser head using the IR sensor. This error originates from the PIC microcontroller and typically requires a device restart.
E02 - IR Marker Error	The system failed to detect the dispenser head using the IR marker. This error originates from the PIC microcontroller and typically requires a device restart.
E11 - Dispenser Error	The system could not detect the dispenser head. Communication with the dispenser failed, and a device restart is required.
E12 - Dispenser Timeout Error	The system did not receive a response from the dispenser head within the expected timeout period. A device restart is required.
E13 - Limit Switch Triggered	A limit switch was triggered unexpectedly, indicating a possible movement boundary collision. The operator should check the head position before resuming operation.
E21 - Parameter Error	One or more tray configuration parameters are outside the valid range. The system highlights the invalid values that need correction.
E22 - Dimension Error	The calculated tray dimensions from the provided parameters exceed the physical tray boundaries. The configuration values must be verified and corrected.
E23 - Skip Values Error	The configured skip positions are invalid for the current tray configuration. The skip settings must be updated in the Advanced settings menu.
E24 - Excessive Skips Error	The total number of configured skip positions exceeds the system's maximum allowed limit. The number of skip positions must be reduced.
E25 - No Valid Positions Error	All positions on the grid have been marked as skipped, leaving no available positions for dispensing. At least one position must be available for operation.
E99 - Unknown Error	An unrecognized error code was encountered. This indicates an error condition not defined in the system's error handling code.

Controller Architecture

Controller Interface

At any point in time, one controller is active on the system, handling the logic of the entire system.

The controller might simply be waiting for user input, or running the stepper motors until a certain position is reached, or waiting for the dispenser to finish a dispense cycle.

Controllers share a common interface, which allows them to interact with the rest of the system in a predictable way.

Controllers can signal that another controller should take over by using the `startNewController` callback.

Controller Methods

Method	Caller	Description
`onStart`	ControllerManager	Triggers initial logic when controller becomes active
`onInteraction`	InteractionHandler	Handles button presses and PLC commands
`onStep`	Main Loop	Runs stepper motors, processes dispenser commands/responses, handles internal state machine

Controller Dependencies

Controllers receive the following dependencies during initialization:

- `dispenserHead`: Axis movement and dispenser commands
- `profileManager`: Get/update profiles
- `phost`: Update UI display

Controller Lifecycle

1. **Initialization**: Controller is created with injected dependencies (`dispenserHead`, `profile–Manager`, `phost`)
2. **Activation**: `ControllerManager` calls `onStart()` to trigger initial logic
3. **Event Loop**:
    - `InteractionHandler` calls `onInteraction()` for button presses and PLC commands
    - Main loop continuously calls `onStep()` for motor control, dispenser communication, and state management
4. **Transition**: Controller calls `startNextController(<con_id>)` callback to transfer control to another controller

Dispense Cycle Flow

This document describes the dispense cycle managed by the RunController, which orchestrates the entire dispensing operation through a series of stages from initialization to completion. This process includes:

- Setting the vibration level and duration on the dispenser head
- Priming the dispenser head
- Calculating the most efficient path to fill all required positions
- Controlling the Z axis to “dip” the dispenser head into the tray

The dispense cycle is broken down into **stages**, to ensure that the system can pause and resume the cycle at any point, and to allow for proper error handling.

Stages Overview

The dispense cycle consists of 11 stages that execute sequentially:

1. **Idle** - Initial idle state, all motors stopped
2. **Set Vibration Level** - Configure vibration level on dispenser
3. **Set Vibration Duration** - Configure vibration duration on dispenser
4. **Zero** - Move all axes to zero position (home)
5. **Start Prime** - Send prime command to dispenser
6. **Wait Prime** - Wait for prime cycle to complete
7. **Move** - Move XY axes to target position
8. **Lower Head** - Lower Z axis to dip depth
9. **Start Dispense** - Send dispense command to dispenser
10. **Wait Dispense** - Wait for dispense cycle to complete
11. **Raise Head** - Raise Z axis back to zero position

Error Handling

During any stage, the following errors will trigger a dialog and cause certain actions:

Error Code	Description
E01 - IR Sensor Failure	Pause cycle indefinitely and request user to restart the device
E02 - Marker Not Detected	Pause cycle indefinitely and request user to restart the device
E11 - ACK Error	Pause cycle indefinitely and request user to restart the device
E12 - Cycle Timeout	Pause cycle indefinitely and request user to restart the device
E13 - Limit Switch Triggered	Pause cycle and allow user to resume or stop the dispense cycle

When paused, the user can:

- **Resume**: Continue from the current stage
- **Stop**: Abort the cycle and return to home position

Skipping Mechanism

This document describes the skipping mechanism that allows users to exclude specific positions from the dispense cycle. The system supports three types of skip definitions:

- **Skip Columns** - Exclude entire columns from dispensing
- **Skip Rows** - Exclude entire rows from dispensing
- **Skip Individual Positions** - Exclude specific cells by their coordinates

The skipping mechanism is configured through the Advanced Settings screen and is validated to ensure the skip parameters are within acceptable limits.

Skip Types

Skip Columns

Users can specify one or more column indices to skip during the dispense cycle. Columns are specified as comma-separated values.

Example: `C1,C5,C10` will skip columns 1, 5, and 10.

Maximum: Up to 42 columns can be skipped (defined by `MAX_SKIP_POSITIONS_COLUMNS` in `Constants.h`).

Skip Rows

Users can specify one or more row indices to skip during the dispense cycle. Rows are specified as comma-separated values.

Example: `R2,R7,R15` will skip rows 2, 7, and 15.

Maximum: Up to 33 rows can be skipped (defined by `MAX_SKIP_POSITIONS_ROWS` in `Constants.h`).

Skip Individual Positions

Users can specify individual cell positions to skip using coordinate notation. Each position is specified as `(column,row)` and multiple positions are separated by semicolons.

Example: `C3R4,C8R12,C15R20` will skip cells at column 3 row 4, column 8 row 12, and column 15 row 20.

Maximum: Up to 30 individual positions can be skipped (defined by `MAX_SKIP_POSITIONS_INDIVIDUAL` in `Constants.h`).

Total Skip Limit

The system enforces a **total maximum of 100 skip positions** across all three skip types combined (defined by `MAX_SKIP_POSITIONS_TOTAL` in `Constants.h`). This limit (and the individual limits) primarily arises from limited memory resources for the string buffer used to store the skip parameters, and the limited storage space available in the microcontroller's EEPROM.

Calculation

The total skip count is calculated as:

```
Total Skips = (Number of Skipped Columns) + (Number of Skipped Rows) +
(Number of Individual Positions)
```

Note: When a column or row is skipped, it counts as a single skip position regardless of how many cells it contains. Individual cell skips each count as one position.

PLC Serial

Overview

This guide explains how to communicate with the system using PLC serial commands. The system uses a simple, fixed-length binary protocol over a serial connection for reliable command transmission.

Hardware Setup

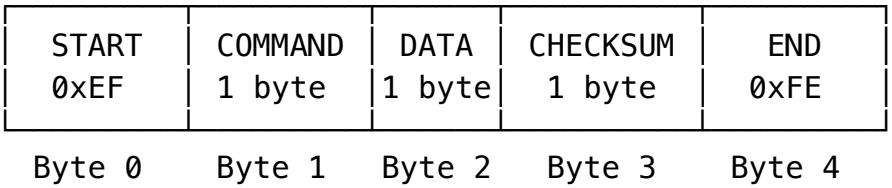
Connection Requirements

- **Baud Rate:** 19200
- **Data Format:** 8 data bits, 1 stop bit, no parity (8N1)
- **Flow Control:** None

Message Protocol

Message Structure

Every message consists of exactly **5 bytes** in the following format:



Byte	Field	Value	Description
0	START_BYTE	`0xEF`	Fixed start delimiter
1	COMMAND	Variable	Command code (see command table)
2	DATA	Variable	Command parameter (use `0x00` if not needed)
3	CHECKSUM	Calculated	Sum of COMMAND + DATA (masked to 8 bits)
4	END_BYTE	`0xFE`	Fixed end delimiter

Checksum Calculation

The checksum ensures message integrity:

```
CHECKSUM = (COMMAND + DATA) & 0xFF
```

Example: - COMMAND = `0x30` - DATA = `0x00` - CHECKSUM = `(0x30 + 0x00) & 0xFF` = `0x30`

Available Commands

Command Reference Table

Command	Code	Data Byte	Hex Message	Description
START	0x30	0x00	EF 30 00 30 FE	Start system operation
STOP	0x31	0x00	EF 31 00 31 FE	Stop system operation
PAUSE	0x32	0x00	EF 32 00 32 FE	Pause current operation
RAISE_Z	0x42	Variable	EF 42 XX CS FE	Raise Z-axis (XX = value in mm)
LOWER_Z	0x41	Variable	EF 41 XX CS FE	Lower Z-axis (XX = value in mm)

System Responses

Acknowledgment (ACK)

When a valid command is received and accepted, the system **mirrors back** the exact same 5-byte message:

Example:

```
PLC sends:     EF 30 00 30 FE  (START command)
System replies: EF 30 00 30 FE  (ACK – same message)
```

Negative Acknowledgment (NAK)

The system sends a NAK response for: - **Invalid checksum:** Checksum does not match calculated value - **Unknown command:** Command byte is not in the supported command list

NAK Response Format:

```
15 15 15 15 15  (five NAK bytes: 0x15)
```

Action: Verify message format and checksum, then retransmit.

Busy Response (BUSY)

If a START command is sent while the system is busy, the system responds with:

```
16 16 16 16 16  (five BUSY bytes: 0x16)
```

Action: Wait for system to complete current operation before retrying.

No Response

The system will **not respond** if: - Message format is invalid (wrong start/end bytes) - Message length is not exactly 5 bytes - First byte is not the START_BYTE

Action: Verify message format and retransmit.

System Status Messages

COMPLETED Message

When the system completes an operation, it sends:

```
EF 17 00 17 FE
```

Breakdown: - START: `0xEF` - COMMAND: `0x17` (completion indicator) - DATA: `0x00` - CHECKSUM: `0x17` - END: `0xFE`

Usage: Monitor for this message to know when the system has finished processing.
